

Data Manipulation & Preprocessing

Advanced Machine Learning

Sam Ogden

Learning objectives

- Move between NumPy arrays and PyTorch tensors without friction
- Predict tensor shapes through indexing, reshaping, and combining
- Turn a pandas DataFrame into a clean, model-ready tensor
- Spot preprocessing steps that leak information across splits

From raw data to a model

- Every model starts the same way: **raw data** → **tensor** → **model**
- Most of the bugs you'll hit early on are shape bugs, not math bugs
- Today: get fluent moving between numpy, pandas, and torch tensors

NumPy → PyTorch

numpy.ndarray vs torch.Tensor

What's the same

- Both are n-dimensional arrays of numbers
- Same mental model: shape, dtype, indexing, broadcasting
- Mostly the same method names too

What's different

- torch defaults to `float32`, numpy to `float64`
- torch tensors can live on a GPU; numpy arrays can't
- torch can track gradients if you ask it to (`requires_grad`) — more on this in few days

Creating one looks nearly identical

NumPy

```
1 import numpy as np
2
3 a = np.arange(12)
4 print(a)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

PyTorch

```
1 import torch
2
3 t = torch.arange(12)
4 print(t)
```

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```


Indexing works the same way

NumPy

```
1 print(a[2:5])
```

```
[2 3 4]
```

PyTorch

```
1 print(t[2:5])
```

```
tensor([2, 3, 4])
```

Reshaping works the same way

NumPy

```
1 print(a.reshape(3, 4))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

PyTorch

```
1 print(t.reshape(3, 4))
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```


Same values, different default dtype

NumPy

```
1 print(np.ones(3).dtype)
```

float64

PyTorch

```
1 print(torch.ones(3).dtype)
```

torch.float32

- numpy defaults to `float64` — general scientific computing, precision first
- torch defaults to `float32` — half the memory, and GPUs are much faster at it

`np.arange(12)` and `torch.arange(12)` don't give you the same dtype by default. Be explicit when it matters:

```
1 t32 = torch.arange(12, dtype=torch.float32)
2 print(t32.dtype)
```

torch.float32

The explicit connection between the two

NumPy → PyTorch

```
1 shared = torch.from_numpy(a)
2 shared[0] = 99
3 print(a[0])
```

99

PyTorch → NumPy

```
1 back = t.numpy()
2 back[0] = -1
3 print(t[0])
```

tensor(-1)

Both directions share memory — no copy happens. Change one, change the other. Great for speed, easy to get burned by if you're not expecting it.

Foreshadowing: `requires_grad`

```
1 w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
2 print(w)
```

```
tensor([1., 2., 3.], requires_grad=True)
```

- `requires_grad=True` tells torch to track every operation done on this tensor
- That tracking is what makes automatic differentiation possible
- This is the thing numpy just doesn't do — we'll build directly on it when we get to training loops

Check your understanding

```
x = torch.arange(6, dtype=torch.float64)
```

Predict:

- `x.dtype`
- `x.numpy().dtype`
- What happens to `x` if you modify `x.numpy()` in place?
- Why did we write `dtype=torch.float64` here instead of just letting it default? When might you actually want `float64`?

Answers:

- `x.dtype` → `torch.float64`
- `x.numpy().dtype` → `float64` — numpy mirrors torch's dtype here
- Modifying `x.numpy()` in place also modifies `x` — they share memory
- `float64` earns its cost when precision loss would actually break something — accumulating many small numbers, ill-conditioned linear algebra, scientific/financial computation. Rare in DL training itself; more common in preprocessing or evaluation code

Shape manipulation

The shape is the *contract*

- Shape encodes what the data *means*: which axis is batch, which is channel, which is a feature
- Most early bugs in this class are shape bugs, not math bugs
- Before you run a line, predict its output shape — that habit is worth more than memorizing the API

Reshape: reinterpreting the same data

```
1 x = torch.arange(1, 10)
2 print(x)
3 print(x.shape)
4
5 square = x.reshape(3, 3)
6 print(square)
7 print(square.shape)
```

```
tensor([1, 2, 3, 4, 5, 6, 7, 8, 9])
torch.Size([9])
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
torch.Size([3, 3])
```

- Same 9 numbers, **same underlying memory**, it's just a different shape way of thinking about the data.
 - Think a 2D array
- Reshape doesn't move or copy data; it changes how the data is *read*

The same idea, at model scale

```
1 x = torch.arange(64 * 1 * 28 * 28).reshape(64, 1, 28, 28)
2 print(x.shape)
3
4 flat = x.reshape(x.shape[0], -1)
5 print(flat.shape)
```

```
torch.Size([64, 1, 28, 28])
torch.Size([64, 784])
```

- A batch of 64 grayscale 28×28 images, flattened to feed an neural network
- `-1` means “figure this dimension out for me”
 - useful when you don’t want to hardcode it

cat joins along an existing axis, **stack** makes a new one

torch.cat

```
1 a1 = torch.zeros(2, 3)
2 a2 = torch.ones(2, 3)
3 print(torch.cat([a1, a2], dim=0).shape)
```

torch.Size([4, 3])

torch.stack

```
1 print(torch.stack([a1, a2], dim=0).shape)
torch.Size([2, 2, 3])
```

- **cat** on two (2, 3) tensors along **dim=0** → (4, 3) — no new axis, just longer
- **stack** on the same tensors → (2, 2, 3) — a brand new axis is created

squeeze / unsqueeze: adding or removing size-1 axes

```
1 example = torch.arange(5)
2 print(example.shape)
3
4 batched = example.unsqueeze(0)
5 print(batched.shape)
```

torch.Size([5])

torch.Size([1, 5])

- A model is built to expect a **batch** of examples, shape `(batch, features)` — even when you only have one example to give it
- `unsqueeze(0)` makes a single example *look like* a batch of one, so it fits the model's expected shape
- `squeeze(0)` undoes it — strip the batch axis back off once you're done

Broadcasting: the rule that lets shapes differ

Compare shapes from the *last* dimension backward. Two dimensions are compatible if they're equal, or one of them is 1.

```
1 batch = torch.ones(4, 3)
2 bias = torch.tensor([1.0, 2.0, 3.0])
3 print((batch + bias).shape)
```

```
torch.Size([4, 3])
```

- $(4, 3)$ and $(3,)$ line up: $3 == 3$, then the missing dimension is treated as 1
- `bias` gets applied to every row without you writing a loop or copying it 4 times

When broadcasting can't save you

```
1 batch = torch.ones(4, 3)
2 wrong = torch.tensor([1.0, 2.0])
3 batch + wrong
```

RuntimeError

Traceback (most recent call last)

Cell In[19], line 3

```
1 batch = torch.ones(4, 3)
2 wrong = torch.tensor([1.0, 2.0])
----> 3 batch + wrong
```

RuntimeError: The size of tensor a (3) must match the size of tensor b (2) at non-singleton dimension 1

(4, 3) vs (2,) — 3 and 2 don't match, and neither is 1. This is the most common shape error you'll hit in this class. Read the error, find the two shapes, find the axis that disagrees.

Predict before you run

```
x = torch.arange(24).reshape(4, 6)
y = x[:, :3]
z = torch.stack([y, y], dim=0)
w = z.reshape(z.shape[0], -1)
```

Predict the shape of **x**, **y**, **z**, and **w** — in that order — before running anything.

```
1 x = torch.arange(24).reshape(4, 6)
2 y = x[:, :3]
3 z = torch.stack([y, y], dim=0)
4 w = z.reshape(z.shape[0], -1)
5 print(x.shape, y.shape, z.shape, w.shape)
```

```
torch.Size([4, 6]) torch.Size([4, 3]) torch.Size([2, 4, 3]) torch.Size([2, 12])
```


